

基本ブロックとフローグラフ — 最適化の地図を作る

今日のゴール

アセンブリを**基本ブロック**に切り分け、制御の流れを辺で結んで**フローグラフ**を作る。
Graphviz で図にして、自分のコンパイラが出したコードの形を目で確かめる。

1 この回の位置づけ

- 最適化発展シリーズの第 2 回。前提はコマ 16(完成した `final/mycc.py`)
- この回だけは命令数が減らない。道具を作る回である
- 作ったフローグラフを、以降の回がそのまま使う

回 何に使うか

04 生存変数解析 (ブロックとブロックの間で情報を伝える)

06 コピー伝播・死コード除去

07 ブロック整列とループ回転 (後方辺を探す)

2 なぜ「ブロック」が要るのか

03(命令選択) の `is_dead_after` で、こういう妥協をした。

見つけたもの	判定
<code>reg</code> を読む命令	まだ使われている → 畳めない
<code>reg</code> に書く命令	読まれる前に上書きされた → 畳んでよい

見つけたもの	判定
ラベル・分岐・ <code>call</code>	ここで基本ブロックが終わる → 判断できない

`is_dead_after` は前向きに 1 行ずつ見るだけなので、ラベルに出会うと止まる。ラベルの先は別の経路から飛んでくるかもしれないので、単純な走査では「このレジスタが後で使われるか」を答えられない。

答えるには、「どこからどこへ制御が移りうるか」の地図が要る。それがフローグラフである。

3 基本ブロックとは

途中から入らず、途中から出ない命令の並びのこと。先頭に入ったら、必ず最後まで実行される。

だからブロックの中では「上から順に 1 回ずつ実行される」と考えてよい。考えるべき経路が減るので、解析がぐっと簡単になる。

4 リーダを見つける

ブロックの先頭になる命令を**リーダー**という。規則は 3 つだけである。

規則	いまの出力での例
(a) 最初の命令	関数の先頭
(b) ラベルが指す命令	<code>.L2:</code> の次 (どこから飛んでくる)
(c) 分岐・ジャンプ・ <code>ret</code> の次の命令	<code>beqz a0, .L4</code> の次

重要

`call` はリーダーを作らない。呼び出しは戻ってくるので、制御はそのまま次の命令へ続く。だからブロックを切る必要がない。

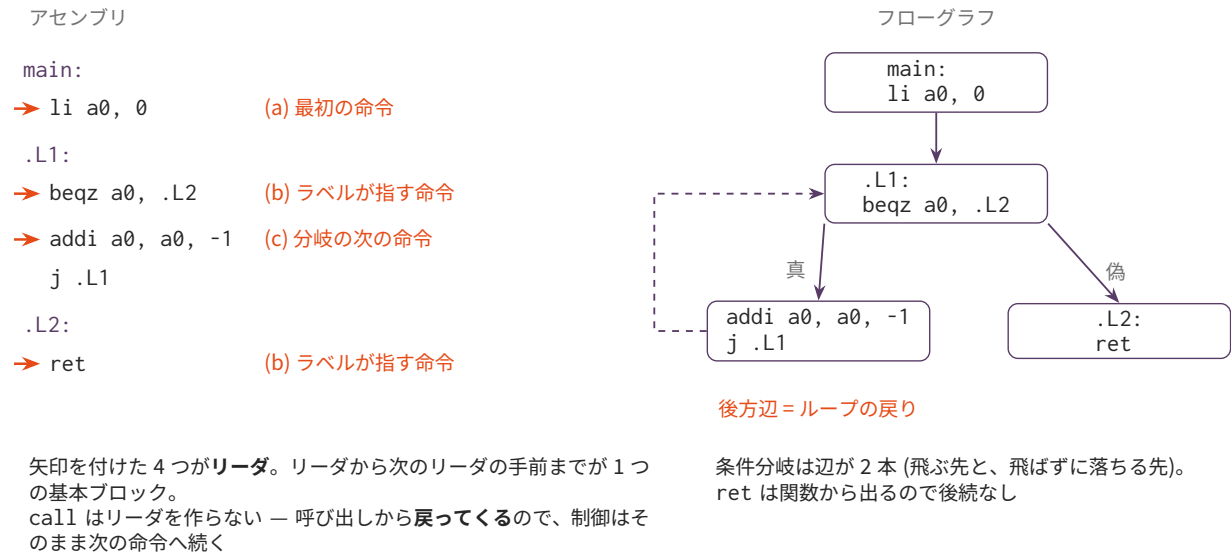


図 1 リーダを見つけて基本ブロックに切り、辺を張る

ただし `call` はレジスタを壊す (`a0 – a7, ra, t0 – t6`)。これは O4 の生存解析で効いてくる別の話である。

5 辺を張る

各ブロックの**最後の命令**だけを見れば、次に何が起きるかが決まる。

最後の命令	後続
<code>ret</code>	なし (関数から出る)
<code>j L</code>	<code>L</code> のブロックだけ
<code>beqz rX, L</code> などの条件分岐	<code>L</code> のブロックと、次のブロックの 2 つ
それ以外	次のブロック (次がリーダだったので切れただけ)

条件分岐で辺が 2 本になるところが要点である。プログラムの「分かれ道」が、そのままグラフの分岐になる。

6 後方辺 ループの正体

自分より前のブロックへ戻る辺を**後方辺**という。これがループである。

```
.L2:  条件          ← ここへ戻ってくる
      beqz a0, .L4
      本体
.L3:  更新
      j .L2          ← 後方辺
.L4:
```

グラフの上では、.L3 のブロックから .L2 のブロックへ戻る辺として現れる。ループを「ソースコードの**while**」ではなく「グラフの後方辺」として見つけられることが、この回のいちばんの収穫である。07 のループ回転は、ここを狙う。

7 実装

Step	実装対象	内容
1	<code>find_leaders</code>	上の 3 つの規則
2	<code>build_blocks</code>	リーダの位置で切る
3	<code>build_edges</code>	最後の命令を見て後続を決める

`Block` クラス、前処理の `strip_asm`(命令だけ取り出してラベルの飛び先を表にする)、`to_dot`(Graphviz 出力) はスケルトンで用意してある。

```
python3 ../optcc.py --passes ' ' ../01_measure/bench/loop_sum.c # 素の出力を見る
python3 golden.py # 図を書き出す
```

8 テスト

```
python3 check.py      # リーダ・ブロック・辺の単体テスト（未実装は SKIP）
python3 golden.py     # bench/*.c のフローグラフを検証して図にする
```

`golden.py` は数を数えるだけでなく、構造の辻褄も確かめる。「`ret` に後続がある」「条件分岐の辺が 1 本しかない」といった間違いは、ここで落ちる。

9 測ってみると

教員の実装ではこうなる（最適化なしの出力に対して）。

ベンチマーク	関数	ブロック	辺	後方辺	最大ブロック	
bubble		1	14	17	3	72
fib_rec		2	6	5	0	34
loop_sum		1	6	6	1	18
matmul		1	20	24	5	103
strops		3	18	19	3	37
合計			64	71	12	

`fib_rec` だけ後方辺が **0** である。ループを 1 つも使っておらず、再帰だけで書かれているからである。O7 のループ回転が `fib_rec` に効かないのは、この時点で予言できる。

`matmul` の最大ブロックは **103 命令**もある。3 重ループの本体が 1 本道だからで、この中には分岐が 1 つも無い。「ブロックの中は素直」という感覚をつかんでおくとよい。

10 発展課題

1. **call** でブロックを切ってみる: 切ると何が変わるか。ブロック数はどれだけ増えるか。なぜ普通は切らないのか
2. 到達しないブロックを探す: 入口から辿れないブロックがあれば、それは死んだコードである。いまのコンパイラは出すか
3. 支配関係 (**dominator**): 「B に行くには必ず A を通る」という関係を計算する。これがあると後方辺を正確に定義できる (いまは「前へ戻る辺」で近似している)
4. ループの入れ子を数える: `matmul` の 3 重ループは、後方辺の包含関係として見える。最も内側のループはどれか

5. 関数ごとに切り出す: `blocks_of(blocks, func)` を使って、関数単位で図を出す

補足

コラム: 地図が無いと最適化は臆病になる。 O3 の `is_dead_after` は、ラベルを見つけると「分からない」と答えて畳み込みをあきらめた。安全ではあるが、機会を逃している。フローグラフがあれば「そのラベルへ飛んでくるのはどのブロックか」が分かるので、もっと正確に答えられる。

もっとも、**正確にすれば必ず得をする、とは限らない。** O3 の畳み込みが正確な解析でどれだけ増えるかは、O6 で実際に測る。最適化の議論は、いつも「測ってみると」から始まる。